

Relatório de Evolução — Sprint 03

EV Challenge — GoodWe · Agentes de IA e Evolução Conversacional

Disciplina: Prompt and Artificial Intelligence · Ciência da Computação — 1º ano · Turma 1CCPW

Projeto: **ChargeGrid Intelligence** — chatbot de gestão de eletropostos

Nome	RM	Turma
Arthur Micarelli Domingos	RM571476	1CCPW
Enzo Yudi de Oliveira Hino	RM570173	1CCPW
Felipe Elze	RM572024	1CCPW
Inaldo Pereira Freitas	RM569672	1CCPW
Henrique Eduardo da Silveira	RM571803	1CCPW

1. Resumo da evolução

Sprints 1 e 2: definição do problema (operador de eletroposto que precisa de respostas rápidas sobre consumo, cobrança e desempenho), persona, fluxo e system prompt (Sprint 1). Na Sprint 2 isso virou um notebook funcional com chamadas manuais à API da OpenAI: um *Orquestrador* (gpt-4o-mini) classificava a pergunta em um dos 10 tipos de intenção e um *Agente Principal* (gpt-4o-mini) recebia os dados dos CSVs mockados injetados no prompt. O histórico era uma lista Python única, limitada a 20 mensagens.

Sprint 03: o núcleo foi refatorado com **LangChain**. Foram adicionados: agente com 9 Tools (create_agent), memória por session_id gerenciada pelo framework, um guardrail de segurança antes do agente que classifica cada mensagem em uma categoria, configuração de dois modelos via .env (MODEL_A/MODEL_B), correção do bug de data, extração do núcleo para chatbot_core.py (notebook e testes usam o mesmo código) e um script único de testes (testes/testes.py) com testes do guardrail, funcionais, de memória, de segurança e de latência, rodando nos dois modelos.

2. Refatoração

Framework escolhido e motivo

LangChain. A escolha segue o que já tinha sido planejado na Sprint 1, onde LangChain foi indicado para gerenciar contexto, memória de conversa e fluxos. Como o projeto já usava a OpenAI, a integração langchain-openai permitiu trocar o modelo apenas pelo nome.

Principais decisões técnicas

Decisão	Como ficou na Sprint 03
Orquestração	O classificador manual de 10 intenções foi substituído por 9 Tools (@tool): consumo, picos, usuários, cobrança, previsão, desempenho, financeiro, GoodWe e saudação. O próprio agente decide qual Tool chamar e pode usar mais de uma por pergunta.
Memória	RunnableWithMessageHistory + InMemoryChatMessageHistory, com um histórico separado por session_id.
Segurança	Guardrail separado (gpt-4o-mini, temperature 0) com saída estruturada (with_structured_output + Pydantic). Classifica a mensagem em PERMITIDO, PROMPT_INJECTION, FORA_DO_ESCOPO, JURIDICO, FINANCEIRO_PESSOAL ou ELETRICO_PERIGOSO, considerando as últimas 6 mensagens da sessão. Cada bloqueio tem resposta própria indicando o profissional adequado; só PERMITIDO chega ao agente.
Dados	Mesmos CSVs da Sprint 2. O “mês atual” passou a ser a data mais recente do próprio dataset, e não a data do computador.
Modelos	Agente com temperature 0.4 e max_tokens 600 (mesmos valores da Sprint 2). Modelos testados: gpt-4o-mini e gpt-5-nano.

Trade-offs identificados

- **Latência e custo:** o guardrail adiciona uma chamada extra à LLM em todo turno (latência média de 3,01 s com o gpt-4o-mini).
- **Memória volátil:** o histórico fica em memória e se perde ao reiniciar o processo.
- **APIs depreciadas:** na execução, o LangChain avisou que RunnableWithMessageHistory e

InMemoryChatMessageHistory estão depreciados e recomendou a persistência do LangGraph.

• **Guardrail como ponto único:** se ele errar para mais, bloqueia perguntas legítimas (ocorreu na primeira versão, ver seção 4); por isso adotou-se a regra “na dúvida, PERMITIDO”, já que o agente também tem as regras de segurança no system prompt.

3. Comparativo antes × depois

As mesmas 5 perguntas funcionais foram executadas novamente. Resultados da Sprint 2 em testes/resultados.txt (projeto antigo) e da Sprint 03 em testes/resultados.txt, após a correção do guardrail.

Critério	Sprints 1 e 2	Sprint 03
Arquitetura	Manual: Orquestrador + Agente Principal com chamadas diretas à API OpenAI	LangChain: agente com 9 Tools + guardrail por categoria
Memória	Lista Python única, últimas 20 mensagens	Por session_id, gerenciada pelo framework
Modelo	gpt-4o-mini (sem comparação)	gpt-4o-mini, escolhido após teste contra gpt-5-nano
Perguntas funcionais respondidas	5/5 (88,82 kWh no mês; Beatriz Costa com 33,87 kWh; cobrança por sessão; 2 sessões previstas)	5/5 com os mesmos valores (gpt-4o-mini)
Data de referência (“este mês”)	datetime.now(): em 21/09/2026 encontra 0 sessões e 0 kWh	Data mais recente dos dados (14/06/2026): 11 sessões e 88,82 kWh
Memória em 3 turnos	Não havia teste específico	2/2 cenários (Solar Park/12 vagas e CHR-01)
Testes de segurança	Não havia testes de segurança	6/6 adequados, com indicação de profissional quando necessário
Guardrail	Apenas regras no system prompt	17/17 mensagens classificadas corretamente
Latência média	Não medida	3,01 s por pergunta (guardrail + agente)
Tokens por turno	Não medido	Não medido

Resultados por modelo (Sprint 03)

Teste	gpt-4o-mini	gpt-5-nano
Guardrail (usa sempre gpt-4o-mini)	17/17	17/17
Funcionais	5/5	0/5 (respostas vazias)
Memória (3 turnos)	2/2	0/2 completos; 1 parcial (acertou só o turno 3 do Solar Park)
Segurança	6/6	5/6 (caso da especificação técnica vazio)
Latência média	3,01 s	8,23 s

A nova arquitetura tornou o chatbot melhor? Sim. Com o gpt-4o-mini, a Sprint 03 mantém a mesma qualidade funcional da Sprint 2 (5/5, mesmos valores) e acrescenta o que antes não existia: memória por sessão validada em 3 turnos (2/2), resistência a Prompt Injection e comportamento seguro nos temas sensíveis (6/6), guardrail com 17/17 acertos e a correção do bug de data. O custo é uma chamada extra ao modelo por turno, por causa do guardrail.

4. Problemas encontrados e soluções

	Problema 1 — “mês atual” usando a data do computador	Problema 2 — guardrail bloqueando perguntas legítimas
Problema	A Sprint 2 usava datetime.now(). Fora de junho/2026 (mês dos dados mockados), as consultas retornavam vazio sem erro: em 21/09/2026, 0 sessões.	Na primeira execução, o guardrail bloqueou as 5 perguntas funcionais (0/5 respondidas). O prompt mandava bloquear o que estivesse “fora do escopo”, mas não dizia qual era o escopo; a saída era texto livre ALLOWED/BLOCKED; e todo bloqueio tinha a mesma mensagem genérica.

	Problema 1 — “mês atual” usando a data do computador	Problema 2 — guardrail bloqueando perguntas legítimas
Alternativas	Fixar uma data manualmente; regenerar os dados para o mês atual; usar a data mais recente do dataset.	Remover o guardrail e deixar a segurança só no system prompt; ajustar apenas a leitura da string; reescrever o guardrail com escopo explícito e saída estruturada.
Solução adotada	Funções obter_data_referencia/obter_mes_referencia usam a data máxima do CSV.	Prompt descreve o escopo com exemplos; saída estruturada por categoria (Pydantic); resposta específica por categoria; regra “na dúvida, PERMITIDO”; nova seção de teste que mede os acertos do guardrail.
Justificativa	Funciona em qualquer data de execução e com qualquer conjunto de dados, sem manutenção manual. Comprovado no teste: 11 sessões / 88,82 kWh.	Manteve a camada de segurança sem bloquear o uso normal. Resultado: guardrail 17/17, funcionais de 0/5 para 5/5 e segurança 6/6 com indicação do profissional adequado.

	Problema 3 — respostas vazias do gpt-5-nano
Problema	Com a mesma configuração do agente (temperature 0.4, max_tokens 600), o gpt-5-nano retornou resposta vazia em 11 das 12 mensagens que chegaram ao agente, com latência média de 8,23 s.
Alternativas	Aumentar o max_tokens; reduzir o esforço de raciocínio do modelo; testar outro modelo; manter o gpt-4o-mini.
Solução adotada	gpt-4o-mini definido como modelo da versão final (MODEL_A).
Justificativa	Foi o único que passou em todos os testes (5/5, 2/2, 6/6) e foi o mais rápido. A causa das respostas vazias não foi verificada; uma hipótese é o raciocínio interno do modelo consumir o limite de tokens antes da resposta.

Problema 4 (menor) — acesso a modelos: nem toda chave OpenAI tem acesso aos mesmos modelos (erro 403). Alternativas: fixar os modelos no código ou torná-los configuráveis. Solução: MODEL_A/MODEL_B no .env e o comando `python testes/testes.py --listar-modelos`, que lista os modelos disponíveis para a chave.